
12

Best Practices

Contents

■ Background	12-2
■ Best Practices for Assign Service	12-3
■ Exercise 12-1: Create an Assign Service Process	12-4
■ Best Practices for Invoke Sub-Process Service	12-10
■ Exercise 12-2: Using the Invoke Sub-Process Service in Sync Mode	12-12
■ Best Practices for Release Service	12-17
■ Exercise 12-3: Re-executing the Business Process	12-18
■ Exercise 12-4: Adding the Release Service to the BasicInventoryProcess	12-21
■ Lesson Review	12-25

Background

Introduction	This lesson provides you with the best practices that you can adopt when working with Sterling B2B Integrator.
---------------------	--

Lesson objectives	This lesson is designed to help you to: ■ Implement best practices for Assign Service ■ Implement best practices for Invoke Sub-Process Service ■ Implement best practices for Release Service
--------------------------	---

Best Practices for Assign Service

Assign Service vs Assign Activity

- The Assign Service can be used when there are multiple assignments that need to be made consecutively.
 - Reducing the number of steps in a process can increase performance. A recent test with two assign statements versus the same values in an Assign service ran in approximately half the time.
 - The Assign Service can be found in the Sync Mode Stencil.
-

Exercise 12–1 Create an Assign Service Process

- | | |
|---------------------|--|
| Instructions | <ol style="list-style-type: none">1. Open the Graphical Process Modeler.2. Click New.3. Open the Stencils if not already showing. (Be sure to include the Sync Mode Stencil).4. Move the following operations and services to your workspace by dragging the icons from the Stencil to the workspace:<ul style="list-style-type: none">■ Start (1)■ End (1)■ Sequence Start (1)■ Sequence End (1)■ Assign Service■ BP MetaData Service (1)■ XML Encoder (1)■ File System Adapter (1) |
|---------------------|--|

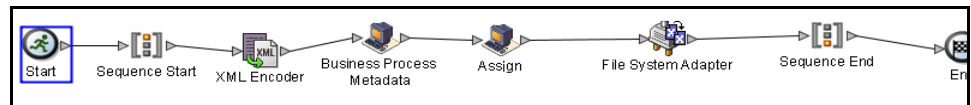
(Continued on next page)

Exercise 12–1 Create an Assign Service Process(Continued)

Connecting Activities

Connecting activities determine the order in which the activities are run within the Business Process. You can follow the steps that are listed to connect the activities.

1. Connect the activities from the previous exercise as shown below.



2. Click the **XMLEncoder** Service to open the Service Editor and perform the following steps:
 - a. Select **XMLEncoder** from the Config Drop-down list box.
 - b. Select **Use existing XML document** from the drop-down list box in the Value column for the mode field.
 - c. Select **Yes** from the drop-down list box in the Value column for the output_to_process_data field.
3. On the Business Process Metadata service choose **BPMetadataInfoService** as the configuration, and accept the defaults for all other parameters.
4. Click **Assign Service** to open the Service Editor, and click **Advanced** to open the Advanced Editor.

The **Advanced Editor** is available in different services and adapters. It will allow you to create your own Assign statements. Some services require you to use the Advanced Editor to create their configuration settings.

(Continued on next page)

Exercise 12–1 Create an Assign Service Process(Continued)

Connecting Activities (Continued)

5. Click **Add** and complete the editor options with the following information:

Name	Value
OrderTotal	sum(INVENTORY/PRODUCT/PRICE) Note: This XPath statement must be typed on a single line and the XPath check box must be selected.
OutputFile	concat('Proc#',//WORKFLOW_ID/text(),'_Total_',//OrderTotal/text(),'.txt') Note: This XPath statement must be typed on a single line and the XPath check box must be selected.

6. Open the **Service editor** for the file system adapter, and choose your file system adapter configuration that you have used for prior exercises like **FS_Test**.

Name	Value
Action	Extraction
assignedFileName	/ProcessData/OutputFile/text() Note: This XPath statement must be typed on a single line and the XPath check box must be checked.
extractionFolder	/home/train/fsext

(Continued on next page)

Exercise 12–1 Create an Assign Service Process(Continued)

**Connecting
Activities
(Continued)**

-
7. Save the Business Process as **AssignService_Test**. Validate and check-in the Business Process.
 8. Execute the business Process with the file **InventoryOf11.xml**.
-

(Continued on next page)

Exercise 12–1 Create an Assign Service Process(Continued)

Obtaining Business Process Results

You can follow the listed procedure to obtain business process results.

1. Click **Business Process > Monitor > Current Processes**. The Business Process Monitor screen displays the current processes.
2. Locate your instance of **AssignService_Test** in the list.
3. Click corresponding **Business Process ID** to view the Business Process Detail page.

(Continued on next page)

Exercise 12–1 Create an Assign Service Process(Continued)

Obtaining Business Process Results (Continued)

- Click the **info** icon in the Instance Data column of the Assign Service to view Process Date.

```
<ITEM>Gizmo K</ITEM>
<PRICE>12</PRICE>
</PRODUCT>
</INVENTORY>
<BPDATA>
  <WORKFLOW_ID>27373</WORKFLOW_ID>
  <MESSAGE_FROM_SERVICE>admin</MESSAGE_FROM_SERVICE>
  <WFD_ID>825</WFD_ID>
  <WFD_VERSION>4</WFD_VERSION>
  <WFD_NAME>AssignService_Test</WFD_NAME>
  <WFD_DESCRIPTION>alpha order</WFD_DESCRIPTION>
  <WFD_STATE>ACTIVE</WFD_STATE>
  <WFD_STATUS>SUCCESS</WFD_STATUS>
  <WFD_TYPE>NORMAL</WFD_TYPE>
  <WFD_PRIORITY>4</WFD_PRIORITY>
  <WFD_PERSISTENCE_LEVEL>FULL</WFD_PERSISTENCE_LEVEL>
  <WFD_LIFE_SPAN>2880 Minute(s)</WFD_LIFE_SPAN>
  <WFD_STORAGE_TYPE>DEFAULT</WFD_STORAGE_TYPE>
  <WFD_RECOVERY_LEVEL>DEFAULT</WFD_RECOVERY_LEVEL>
  <WFD_DOC_TRACKING_FLAG>false</WFD_DOC_TRACKING_FLAG>
  <WFD_DEADLINE_INTERVAL>-1</WFD_DEADLINE_INTERVAL>
  <WFD_EVENT_LEVEL>FULL</WFD_EVENT_LEVEL>
</BPDATA>
<OrderTotal>191</OrderTotal>
<OutputFile>Proc#27373_Total_191.txt</OutputFile>
</ProcessData>
```

Important!

Notice the OrderTotal and OutputFile tags in Process Data from the Assign Service. This requires two Assign activities.

- Check the **/home/student/fsx** directory for the output file.

Best Practices for Invoke Sub-Process Service

How the Invoke Sub-Process Service Works

When the Invoke Sub-Process service is set to synchronous mode, the parent suspends processing until it receives data from the child. In synchronous mode, the parent is notified when the child encounters errors.

When the Invoke Sub-Process service is set to asynchronous mode, the parent and child process data simultaneously and independently of each other. Therefore, the parent does not receive notification when the child encounters errors.

When the Invoke Sub-Process service is set to run a subprocess inline, the subprocess runs as part of the parent process, sharing process data.

When the Invoke Sub-Process service is set to run in embedded mode, the subprocess runs without persistence, meaning that no record of the process is recorded in Application and no tracking is done.

Note: The **Invoke Sub-Process** and **Invoke Business Process Service** are different names for the same service used to start a child business process.

What Mode Should I Use?

- Asynchronous mode is the default selection and typically used in most applications as it allows for concurrent (multi-threaded) processing of multiple documents.
- Synchronous can be used when you want the child to report back up to the parent or if you want a single threaded process flow. An example of this may be large files such as Catalogs (832) or Product Activity Data (852) where due to the file size, handling multiple files at once can put an undue strain on the environment.
- Inline is a good choice when dealing with high volumes of processes within the system as it minimizes the number of threads that are created for the overall process flow.

(Continued on next page)

Best Practices for Invoke Sub-Process Service (Continued)

Performance Tips

By default, when you use the Invoke Sub-Process service in a business process, all process data passes from the parent process to its subprocess.

However, if you are using the Invoke Sub-Process service in sync mode, a special tag called ``message_to_child/message_to_parent'` allows you to pass along only the ``message_to_child/message_to_parent'` node in the process data of the parent process or subprocess. Using this tag can provide significant performance improvement.

Before invoking a subprocess, create a special tag called ``message_to_child'` in the parent process, and append all of the data needed in the subprocess under this tag. The Invoke Sub-Process service will pass only this tag to the subprocess.

The `'message_to_parent'` tag accomplishes the same task, but limits the amount of data that is returned to the parent process. Only data under the `'message_to_parent'` tag will be passed back to the parent.

Exercise 12–2 Using the Invoke Sub-Process Service in Async Mode

Instructions

You can use the Invoke Sub-Process Service in sync mode in the following manner.

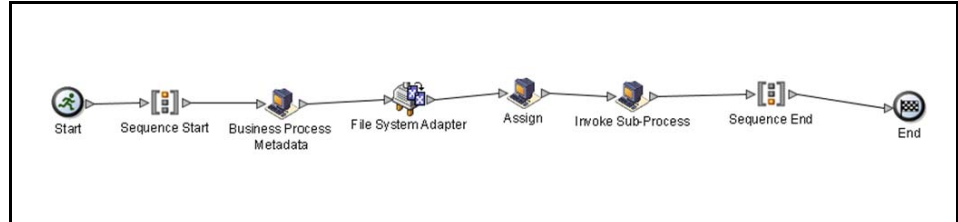
1. Open the **Graphical Process Modeler**.
2. Click **New**.
3. Open the Stencils if not already showing.
4. Move the following operations and services to your workspace by dragging the icons from the Stencil to the workspace:
 - (1) Start
 - (1) End
 - (1) Sequence Start
 - (1) Sequence End
 - (1) Assign Service
 - (1)BP MetaData Service
 - (1) Invoke Sub-Process
 - (1) File System Adapter

(Continued on next page)

Exercise 12–2 Using the Invoke Sub-Process Service in Async Mode (Continued)

Instructions (Continued)

5. Connect the activities from the previous exercise as shown in the following figure.



6. Connecting activities determine the order in which the activities are run within the Business Process.
7. On the Business Process Metadata service choose **BPMetadataInfoService** as the configuration, and accept the defaults for all other parameters.
8. Open the **Service editor** for the file system adapter, and choose your file system adapter configuration.

Name	Value
Action	Collection
collectionFolder	/home/student/fscoll

(Continued on next page)

Exercise 12–2 Using the Invoke Sub-Process Service in Async Mode (Continued)

**Instructions
(Continued)**

9. Click **Assign Service** to open the Service Editor, and click **Advanced** to open the Advanced Editor.
10. Click **Add** and complete the editor options with the following information:

Name	Value
message_t o_child	//WORKFLOW_IDI//PrimaryDocument Note: This XPath statement must be typed on a single line and the XPath check box must be checked.

11. Click the Invoke Sub-Process Service to open the Service Editor and perform the following tasks:
 - a. Select **InvokeSubProcessService** from the Config. Drop-down list box.
 - b. Select **Async** for the Invoke Mode.
 - c. Select your **BasicInventoryProcess** for the WFD_NAME.
12. Save the Business Process as **InvokeSubProcess_Test**. Validate and check-in the Business Process.
13. Paste the **InventoryOf11.xml** file into the /home/train#/fscoll directory.
14. Execute the **InvokeSubProcess_Test** process without input file.

(Continued on next page)

Instructor Note: Inform the students that in the expression “//WORKFLOW_IDI//PrimaryDocument”, the “I” is a pipe symbol.

Exercise 12–2 Using the Invoke Sub-Process Service in Async Mode (Continued)

Obtaining Business Process Results

You can follow the steps that are listed to obtain the business process results.

1. Click **Business Process > Monitor > Current Processes**.
2. The Business Process Monitor screen displays the current processes.
3. Locate your instance of **InvokeSubProcess_Test** in the list.
4. Click the corresponding **Business Process ID** to view the Business Process Detail page.
5. Click the **info** icon in the Instance Data column of Step 4, the Invoke Sub-Process Service.



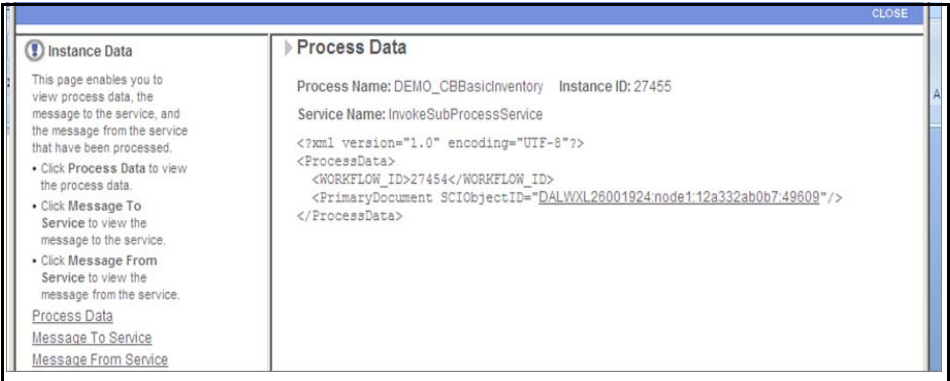
(Continued on next page)

Exercise 12–2 Using the Invoke Sub-Process Service in Async Mode (Continued)

Obtaining Business Process Results (Continued)

Important!	Notice the amount of data in process data. We will only pass the values from the message_to_child tag onto the child process to improve performance.
------------	--

- 6. Close the window to return to the Business Process Detail page.
- 7. Click the **instance ID** of the child process that was invoked, your BasicInventoryProcess.
- 8. Click the **info** icon in the Instance Data column of Step 0, the Invoke Sub-Process Service.



Important!	Notice process data contains only the values from the parent process which are selected.
------------	--

(Continued on next page)

Best Practices for Release Service

Overview The Release Service is a system service that is used to remove extraneous data from the process data. This service comes without predefined parameters, therefore; you must use the Advanced Service Editor to create a parameter that is called TARGET and assign a value to it. The value identifies the process data which is no longer needed. TARGET is the name of the parameter the Release service expects in the "Message TO Service." The value that is assigned to the parameter is an XPath expression which selects the nodes from the Process Data to be released. Even though it is an XPath expression you do not need to check the "Use XPATH" check box.

When dealing with large amounts of Process data, the Release service can be used to clean up portions of Process Data that are no longer needed which aid in the performance of the business process. Multiple Process Data tags can be released within one Release Statement to reduce the overall number of steps in the business process.

Exercise 12–3 Re-executing the Business Process

Instructions

You can re-execute the Invoke_Test process in the following manner.

1. Open the admin console and choose **Business Process > Manager**.
2. Click **Business Process > Manager**. The Business Process Manager screen appears.
3. Type **Invoke** in the Search text box and click **Go!**
4. Click the **Execution Manager** icon for Invoke_Test business Process. **Note:** This is a bp we built earlier in the course.
5. Click the **execute** icon.
6. Click **Browse** and go to **Class Labfiles** to locate the test data. Your instructor will provide the location of the Classlab files.
7. Locate the file **InventoryOf3.xml**, and click **Open**.
8. Click **Go!**
9. Observe the separate Execute Business Process window while the business process runs.
10. Close the Execute Business Process window when the Business Process has completed running.

(Continued on next page)

Exercise 12–3 Re-executing the Business Process(Continued)

Obtaining Business Process Results

You can obtain the business process results in the following manner.

1. Click **Business Process > Monitor > Current Processes**.

The Business Process Monitor screen displays the current processes.

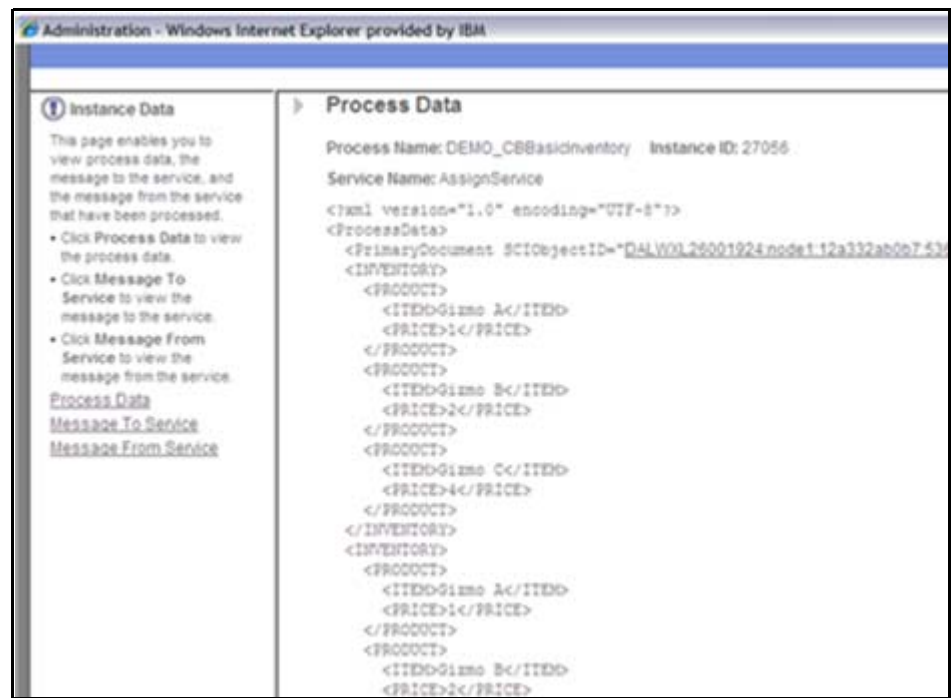
2. Locate your instance of **Invoke_Test** in the list.
3. Click the corresponding **Business Process ID** to view the Business Process Detail page.
4. Click the **Subprocess link** of Step 2 of the process detail. A pop-up window opens with the steps encountered by the child process, BasicInventoryProcess.

(Continued on next page)

Exercise 12–3 Re-executing the Business Process(Continued)

Obtaining Business
Process Results
(Continued)

5. Click the **Info** icon in the Instance Data column of the last Assign Service.



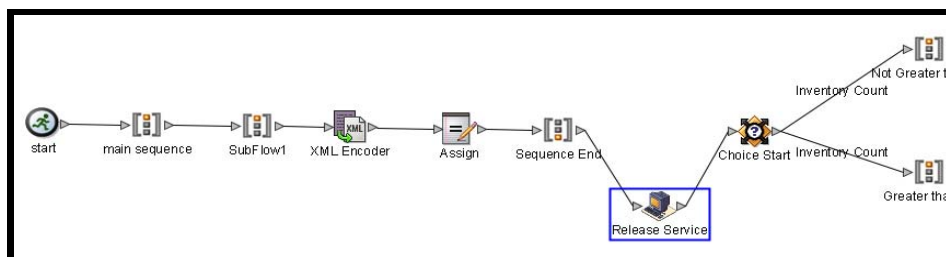
Important!	Notice that the primary document is doubled in process data from being passed from the parent process, which is causing the TotalPrice to be doubled. The release service can be used to remove the first occurrence of primary document.
-------------------	---

Exercise 12–4 Adding the Release Service to the BasicInventoryProcess

Instructions

You can follow the steps that are listed to add the release service to the BasicInventoryProcess.

1. Check out your BasicInventoryProcess.
2. Open the **BasicInventoryProcess** in the Graphical Process Modeler.
3. Insert a **Release Service** in between the Sequence End and Choice Start (as shown in the following figure).



4. Click **Advanced** in the Service Editor for the Release service, and complete the fields in the following table with the given parameters:

Field	Parameter
Name	TARGET
Value	<code> '/ProcessData/INVENTORY[1]' </code> Note: This XPath statement must be typed on a single line and the XPath check box must be checked.

5. Save as **BasicInventoryProcess** and validate.
6. Check in the business process.

(Continued on next page)

Exercise 12–4 Adding the Release Service to the BasicInventoryProcess (Continued)

Testing the Business Process

You can test the business process by following the steps that are listed:

1. Open the admin console and choose **Business Process > Manager**.
2. Click **Business Process > Manager**. The Business Process Manager screen appears.
3. Type **Invoke** in the Search text box and click **Go!**
4. Click the **Execution Manager** icon for the **BasicInventoryProcess** business Process.
5. Click the **execute** icon.
6. Click **Browse** and go to **Class Labfiles** to locate the test data. Your instructor will provide the information on the location of the class lab files.
7. Locate the file **InventoryOf3.xml**, and click **Open**.
8. Click **Go!** Observe the separate Execute Business Process window while the business process runs.
9. Close the **Execute Business Process** window when the execution of the Business Process is completed.
10. Click **Return**.

(Continued on next page)

Exercise 12–4 Adding the Release Service to the BasicInventoryProcess (Continued)

Obtaining Business Process Results

You can follow the steps that are listed to obtain the business process results.

1. Click **Business Process > Monitor > Current Processes**. The Business Process Monitor screen displays the current processes.
2. Locate your instance of **BasicInventoryProcess** in the list.
3. Click the corresponding **Business Process ID** to view the Business Process Detail page.
4. Click the **info** icon in the Instance Data column of step 0. Notice that you see primary document that is written in Process Data.
5. Click the **Info** icon in the Instance Data column of step 1 (the Release Service). Notice that the process data simply contains a link into PrimaryDocument.

(Continued on next page)

Exercise 12–4 Adding the Release Service to the BasicInventoryProcess (Continued)

Obtaining Business Process Results (Continued)

- Click the **Info** icon in the Instance Data column of the last Assign Service. Notice that the Primary Document is in ProcessData with the correct TotalPrice.

The screenshot shows a web application window titled "Business Process Detail" with a "CLOSE" button in the top right corner. The window is divided into two main sections: "Instance Data" on the left and "Process Data" on the right.

Instance Data: This section contains a brief description: "This page enables you to view process data, the message to the service, and the message from the service that have been processed." Below this are three bullet points:

- Click Process Data to view the process data.
- Click Message To Service to view the message to the service.
- Click Message From Service to view the message from the service.

 At the bottom of this section are three links: "Process Data", "Message To Service", and "Message From Service".

Process Data: This section displays XML data for a specific process. It shows:

- Process Name: DEMO_CBBasicInventory
- Instance ID: 27062
- Service Name: AssignService
- XML Content: A snippet of XML starting with `<?xml version="1.0" encoding="UTF-8"?>` followed by `<ProcessData>`. Inside, there is a `<PrimaryDocument SCIObjectID="DALWXL26001924node1:12a332ab0b7-6157"/>` tag, followed by an `<INVENTORY>` block containing three `<PRODUCT>` elements. Each product has an `<ITEM>` and a `<PRICE>` (values 1, 2, and 4). The `<INVENTORY>` block is followed by `<TotalPrice>7</TotalPrice>` and `<Result>Order More Items</Result>`. The entire XML snippet is enclosed in `</ProcessData>` tags.

- Click **Return** from the Business Process Detail window when complete.

Lesson Review

Completed Objectives

This lesson was designed to help you to:

- Implement best practices for Assign Service
 - Implement best practices for Invoke Sub-Process Service
 - Implement best practices for Release Service
-

13

Case Study

Contents

■ Background	13-2
■ Case Study	13-3
■ The Business Problem	13-4
■ Pre-requisites to Complete the Case Study	13-7
■ Case Study Hints	13-9
■ Completing the Case Study	13-12
■ Deliverables Provided by Instructor	13-14
■ Lesson Review	13-15

Background

Introduction

This lesson provides an opportunity to analyze a business problem and create a comprehensive solution to that problem.

Lesson objectives

This lesson is designed to help you to:

- Analyze a business problem
 - Create a comprehensive solution to solve the business problem
-

Case Study

Overview	This Case Study is based on a real implementation and is designed to test your learning in class. The Case Study is planned to take 8 hours (It is intended for flexibility on Friday or earlier if applicable). Because of the complexity of this study, regular stopping points are inserted to bring back the class together for discussion and progress checks. Those falling behind will be brought current by the instructor providing the solution to missing or non-working components.
-----------------	---

Timeline	The following timeline is a guideline established to assist you in managing your time for completing the Case Study. Adjust timing accordingly to pace of class. Examples being starting Fourth Day or Starting 5th Day but going a little longer.
-----------------	--

Fourth Day:

1:00pm - 2:00pm - Discuss and explain the case study; Q&A time lines for the exercise and what services/adapters are required.

2:00pm - 4:00pm - Steps 1-5 from "Steps for Completing the Case Study".

4:00pm - 4:30pm - Breakpoint #1 - Class discussion and debugging of file directories and database table.

Fifth Day:

9:00am - 11:30am - Steps 6-9 from "Steps for Completing the Case Study".

11:30am - 12:00pm - Breakpoint #2 - Class discussion and debugging of ErrRtn, and Output processes, and services/adapters.

12:00pm - 1:00pm - Lunch

1:00pm - 2:30pm Steps 10-13 from "Steps for Completing the Case Study".

2:30pm - 3:00pm - Breakpoint #3 – Class discussion and debugging of Main.

3:00pm - 3:30pm - Final Review/Course Wrap-up/Evals.

The Business Problem

Overview

Historically, the company is operated as two autonomous divisions; one for trucking and logistics, and one for pharmaceuticals. Recently, Sterling B2B Integrator has been purchased and installed, and planned to unite processing into a single system through it. The backend systems will remain independent for the time being; still the evaluation of which ERP software to choose and implement in the future is under discussion.

Company receives inbound data in EDI (X12) format, from Sterling Collaboration Network (our VAN of choice). Some of the data is intended for the company pharmaceutical division, some for the logistics division. Pharma orders must be written to the ORDERS table in the database. Logistics documents must be sent to the logistics application, which requires the data be in EDI format, but with a specific set of delimiters that the app system needs and which the partner will not provide. For data warehousing purposes, must write a copy of the data to a directory, in XML format.

Pull the data from the VAN by way of a communications package. It is being done through FTP communications and it is already established. The files will be transmitted to a folder on the B2Bi server.

Error handling is required, as is hands-free processing.

(Continued on next page)

The Business Problem

(Continued)

Bootstrapped FSA Config Vs. FSA in a BP and CLA2

During this case study the business flow and hints will have you build a business process with a FSA and an Invoke service to start off your process flow, and add error handling if there are no files to collect. Then in the last process you will add a CLA2 to run a script to clear the collection directory.

If limited on time, build a bootstrapped, scheduled FSA configuration as your “starting BP. Then remove the requirement for the CLA2 because the FSA will delete the file on collection

If you have more time try the FSA BP and CLA to clear the directory. Why would someone build all the extra to collect and delete a file? By having a FSA in a BP it gives you more control of the configuration. Such as setting false to “Delete on Collect” and having another service delete the file once processing was complete.

Setting an error on “No files to collect” is not very common as usually it is desired for the FSA to constantly check for incoming files and multiple errors would fill up error reports/logs. This error would be beneficial though, if files were expected at a specific time and if they did not arrive someone should be notified.

(Continued on next page)

The Business Problem

(Continued)

The Business Flow

The Start business process collects data from a directory with an FSA (scheduled). When data is collected, bootstrap Main. If no data is present for collection, invoke ErrRtn business process. This could also be done with a bootstrapped and scheduled FSA configuration instead of a BP but you would lose the error handling.

Main accepts the data handed to it by Start, and encodes it into XML, placing it into ProcessData. Test the data; if it is for the Pharma division, write the PO number, and customer number to the ORDERS table. If it is for the logistics division, change the segment terminator from a "*" to a "=" (equal) and write the altered EDI data out to a processing directory. After this processing is done, pass the results to Output business process. If there are errors anywhere along the line, invoke ErrRtn.

Output accepts the data handed to it by Main, and invokes two paths at the same time; one will call a script outside of Sterling B2B Integrator that will clear the inbound directory, The other moves the XML in Process Data to Primary Document, and write it to a data warehousing directory. If there are errors anywhere along the line, invoke ErrRtn.

ErrRtn is called from multiple processes, as a generic error handling process. Project specifications require that errors be reported to the Sterling B2B Integrator admin via email, requiring the use of an SMTP Send Adapter.

Pre-requisites to Complete the Case Study

Introduction

In order to complete this case study, you will require the following:

1. File System Adapter (2)
2. OnFault activity
3. Invoke Business Process Service
4. XML Encoder Service
5. Choice Start/Choice End
6. Lightweight JDBC Adapter

In order to extract the correct values for use in updating the database in the lightweight JDBC Adapter, you will need to know where the PO_Number and Customer Number are located. If you look at your map you will see the following:

- PO Number = Element 0324 under the BEG segment
- Customer Number = Element 0093 under the N1 segment in the first loop of the Group N1.

Here is an example of an XPath statement finding the Customer number
`//N1[1]/_0093/text()`

(Continued on next page)

Pre-requisites to Complete the Case Study (Continued)

Introduction (Continued)

7. Document Keyword Replace Service

Important!	The Document Keyword Replace Service is not part of your course materials. However, all service and adapter documentation is available on Knowledge Center. There is no default configuration. One must be created before it will show in the GPM
-------------------	---

8. Assign activity

9. Command Line Adapter

Command Line Program: Batch program is supplied in lab files folders.
Program name is Clear_Dir.sh, and will delete all files in directory
c:\Inbound_from_Comms.

10. All Start/All End

11. SMTP Send Adapter

12. Create a table in the Sterling B2B Integrator database to the required specifications

13. Access to product documentation

14. XPath to conduct a choice

15. XPath for DOMToDoc

Case Study Hints

Introduction	The following pages contain some tips that can help you as you work through the case study.
---------------------	---

Additional Information	1. The Start business process contains a File System Adapter and an Invoke Business Process stencil. The FSA should read from Inbound_from_Comms and pass the document to the Main business process. 2. Create a business process called ErrRtn that uses an XML Encoder Service and an SMTP Adapter to send an e-mail (optionally also a note added to Process Data) that there is an error in the business process instance. Use the XML Encoder and xpath to pull from Process Data the Error tag contents and insert into the e-mail. Can also be done with a DOMToDoc. 3. Capture all errors with an OnFault that calls ErrRtn via an Invoke Business Process. 4. Create a map for translating EDI to XML. This map must work for both the Ellis 850 documents and the Unlimited 216 documents. A suggestion is to create an 850 input side map first, then add the 216 segments not there already (PUN, G61, TEM) to the 850 map. Since the map is being used for conversion to XML, data not in the document will be ignored at this step. Here the XML Encoder map will just convert the data it receives. Use 3040 version as previous maps. 5. Create in Main an XMLEncoder stencil, choose to Encode non-XML documents, and use the map you created. a. Set edi_input_element_delimiter to 2A * . b. Set edi_input_segment_delimiter to 7E ~ . c. Set map_name to your combo map. The delimiter settings will allow the map to separate the segments and elements into individual tags instead of grouped together into one tag by the segment. This will make writing your rules and assigns much easier.
-------------------------------	--

(Continued on next page)

Case Study Hints

(Continued)

**Additional
Information
(Continued)**

6. Once the document has been converted into XML in Process Data, create a rule that tests whether the data is from Ellis Labs (N1 contains “ST” and “Ellis”) or from Unlimited (G61 contains “SH” and “CHRISTIAN”). If it’s for Ellis (Pharma), use Assign activities to pull the PO_Num and CustNum values out of the document and into their own Process Data tags. Using a Lightweight JDBC Adapter, write an insert statement that pulls the PO_Num and Cust_Num values from Process Data and inserts them into the table created earlier as CUST_ID and PO_NUM.
7. If the data is for Unlimited (Logistics), call the Document Keyword Replacement Service and replace ~ with = as the segment delimiter, and use an FSA to extract the document to the Altered_Delimiter directory.
8. After the Choice End, call an Invoke Business Process to invoke the Output BP.
9. Output uses an All Start to enact two different paths at the same time. One path uses a Command Line Adapter to call the Clear_Dir.bat script to clear Inbound_from_Comms. The other path uses an XML Encoder to convert Process Data to Primary Document. Use a File System Adapter to write the Primary Document to the Output directory. Configure an OnFault to handle any errors.
10. It is very common in the real world to run business processes to see how things will align in Process Data in order to write correct XPath statements. You do not have to but an example would be to run your map to see the tags and data from the EDI document to write your rules and assigns.

(Continued on next page)

Case Study Hints

(Continued)

11. If you do not set delimiters for your map the following assigns can be helpful in pulling out pieces of information:

a. On an Assign: `substring(_0353/text(), 8, 4)`

It reaches into ProcessData, locates element _0353, counts from the left 8 bytes, and extracts the next 4. Result: The PO number from the data in ProcessData for use in the adapter is extracted.

b. On an Assign: `string-length(_0098/text())`

It reads the overall length of the element in ProcessData and stores that number of bytes as an integer for future use.

c. On an Assign: `substring(_0098/text(), 5, CustNumLen)`

Here, CustNumLen might be the To value from the last Assign. It reaches into the element in ProcessData, places a pointer at the fifth byte, and extracts the rest of the element for use in the adapter.

Completing the Case Study

Overview	In order to successfully complete the Case Study, you must complete the following steps in their given order. 1. Analyze the business problem, plan/sketch a solution. 2. Gather requirements (directories to be created, email address, mail host/port, command-line script location, and function, database table schema). 3. Develop three file system directories (Inbound_from_Comms, Altered_Delimiter, Output_Data). 4. Create database table ORDERS within the Sterling B2B Integrator database. 5. Build the business processes.
To Create the ORDERS Database Table	Database Schema: Table is to be added to Sterling B2B Integrator database in class. Table Name: ORDERS Fields: CUST_ID (string 20), PO_NUM (string 10) Creation Instructions: a. Go to Operations > System > Support Tools > SQL Manager in the UI b. Type create table ORDERS(CUST_ID char(20), PO_NUM char(10)) and press Enter c. Check your work by typing describe ORDERS and click Execute

(Continued on next page)

Completing the Case Study

(Continued)

**To Create the
ORDERS Database
Table
(Continued)**

6. Configure services and adapters Create and check-in ErrRtn.
 7. Test/debug ErrRtn.
 8. Create and check-in Output.
 9. Test/debug Output with and without errors to call ErrRtn.
 10. Create and check-in Main.
 11. Test/debug Main with and without errors to call ErrRtn.
 12. Manually test entire process.
 13. Set schedule for Collection side of FSA or schedule Start BP to test automation.
-

Deliverables Provided by Instructor

Overview

Your Instructor will provide you with the following information to assist you in completing the Case Study.

1. Database table schema.
2. Command-line program.
3. Two EDI interchanges; one for Pharma and one for Logistics.
4. Mail server documentation
5. Documentation on this case study, with breakpoints for discussion.
6. A completed version of this, tested and demonstrable, for discussion and to catch up those lagging behind.

Additional Information

The case study that provided in this lesson is intended to challenge you with a real-world business problem. The examples hear are one way to solve the problem. Feel free to experiment if desired.

If you have any questions during the Case Study or you need assistance, contact the instructor.

Lesson Review

Completed Objectives

This lesson was designed to help you to:

- Analyze a business problem
 - Create a comprehensive solution to solve the business problem
-

